Université Amadou Mahtar Mbow

– UAM –

Polytech Diamniadio

Département science et technologie de l'ingénierie

Master 2 Informatique et Ingénierie

PROJET DE FIN DE MODULE

Rapport rédigé dans le cadre du module d'Apprentissage profond

---

DiagnoCulture

Un système hybride de diagnostic des maladies de la tomate,
fiable et conscient de ses limites

Présenté par

Sidy NDIAYE • **Badara Diop**

Octobre 2026

Résumé

Ce projet conçoit un prototype d'aide au diagnostic des maladies foliaires de la tomate à partir de photos de feuilles. Nous traitons les trois exigences du cahier des charges : reconnaître les 10 classes connues avec une bonne précision, rester léger et rapide pour un smartphone d'entrée de gamme, et savoir rejeter une entrée inconnue au lieu d'inventer un diagnostic. Nous entraînons d'abord deux CNN de zéro (un réseau classique et une version résiduelle), puis nous passons au transfer learning avec deux backbones pré-entraînés. Nous construisons ensuite un auto-encodeur pour la détection d'anomalies, un système hybride de rejet de l'inconnu combinant la confiance du classifieur et l'erreur de reconstruction, une extension générative avec un VAE, et enfin une analyse critique par Grad-CAM. Notre meilleur classifieur (MobileNetV2 fine-tuné complet) atteint une F1-macro de 0,992 au test, et le système hybride permet d'automatiser environ 87% des cas à plus de 95% de fiabilité tout en routant le reste vers un expert.

Table des matières

1 Introduction **3**

2 Données et protocole expérimental **3**
2.1 Jeu de données . . . 3
2.2 Analyse exploratoire et normalisation . . . 4
2.3 Découpage et vérification anti-fuite . . . 5
2.4 Reproductibilité et budget de calcul . . . 5

3 Partie 1 : modèles de base entraînés de zéro **6**
3.1 Méthode . . . 6
3.2 Résultats . . . 6
3.3 Discussion et tableau comparatif . . . 8

4 Partie 2 : transfer learning et choix d'architecture **8**
4.1 Méthode . . . 8
4.2 Résultats . . . 9
4.3 Discussion et recommandation . . . 9

5 Partie 3 : auto-encodeurs et détection d'anomalies **10**
5.1 Méthode . . . 10
5.2 Résultats . . . 10
5.3 Discussion, analyse des échecs . . . 11

6 Partie 4 : système hybride avec rejet de l'inconnu **12**
6.1 Méthode . . . 12
6.2 Résultats . . . 12
6.3 Discussion . . . 13

7 Partie 5 : extension générative (VAE) **14**
7.1 Méthode . . . 14
7.2 Résultats . . . 14

7.3 Discussion 15

8 Partie 6 : analyse critique et responsabilité **15**

8.1 Explicabilité par Grad-CAM 15

8.2 Analyse d'erreurs 16

8.3 Du laboratoire au champ 16

8.4 Enjeux éthiques et sociaux 16

8.5 Limites et perspectives 17

9 Tableau récapitulatif global **17**

10 Conclusion et questions de synthèse **17**

A Déclaration d'usage de l'IA générative **18**

1 Introduction

Au Sénégal, la tomate est une culture maraîchère majeure, notamment dans la zone des Niayes et la vallée du fleuve Sénégal. Les maladies foliaires y causent des pertes importantes, et un diagnostic tardif ou erroné conduit à des traitements inadaptés. Une coopérative agricole nous mandate pour concevoir un prototype d'aide au diagnostic à partir de photos prises au smartphone.

Le cahier des charges comporte trois exigences, qui correspondent aux trois grandes familles de modèles vues en cours. D'abord, reconnaître les maladies connues avec une bonne précision, ce qui relève des réseaux convolutifs et du transfer learning. Ensuite, fonctionner sur un smartphone d'entrée de gamme, ce qui impose un compromis entre précision, nombre de paramètres et latence. Enfin, ne pas se tromper avec aplomb : face à une maladie jamais vue ou à une image qui n'est pas une feuille de tomate, le système doit répondre « inconnu » plutôt que d'inventer un diagnostic, ce qui mobilise les auto-encodeurs, la détection d'anomalies et une approche hybride supervisé / non supervisé.

La question de recherche est donc la suivante : comment combiner un classifieur supervisé et un auto-encodeur non supervisé pour obtenir un système à la fois précis, léger et capable de rejeter ce qu'il ne connaît pas ? Le rapport suit l'ordre des six parties du projet. Après une description des données et du protocole, chaque partie présente sa méthode, ses résultats et sa discussion. La conclusion répond aux six questions de synthèse de l'énoncé, et une annexe déclare notre usage des outils d'IA générative.

2 Données et protocole expérimental

2.1 Jeu de données

Le jeu principal est PlantVillage, sous-ensemble « Tomate » : 10 classes (9 maladies et la classe saine). La version que nous avons téléchargée via `kagglehub` (dataset `emmarex/plantdisease`) contient 16 011 images couleur de feuilles, prises en laboratoire sur fond uniforme. Après la déduplication décrite plus bas, il reste exactement 15 997 images. Le tableau 1 donne les effectifs exacts par classe et par sous-ensemble ; le déséquilibre est net, avec un rapport de 8,6 (3208 contre 373) entre la classe la plus fournie (*YellowLeaf__Curl__Virus*) et la plus rare (*mosaic__virus*).

Table 1 – Effectifs exacts des 10 classes après déduplication, par sous-ensemble (découpage 70/15/15, graine 42).

Classe	Train	Val	Test	Total
Tomato_YellowLeaf_Curl_Virus	2246	481	481	3208
Tomato_Bacterial_spot	1489	319	319	2127
Tomato_Late_blight	1331	285	285	1901
Tomato_Septoria_leaf_spot	1239	266	266	1771
Tomato_Spider_mites	1174	251	251	1676
Tomato_healthy	1109	238	238	1585
Tomato_Target_Spot	982	211	211	1404
Tomato_Early_blight	700	150	150	1000
Tomato_Leaf_Mold	666	143	143	952
Tomato_mosaic_virus	261	56	56	373
Total	**11 197**	**2 400**	**2 400**	**15 997**

2.2 Analyse exploratoire et normalisation

L'analyse exploratoire (notebook `00_eda_split`) confirme le déséquilibre ci-dessus et la bonne qualité des images. La figure 1 montre un exemple de feuille pour chacune des dix classes, et la figure 2 la distribution des effectifs. On voit que toutes les images partagent un **fond de laboratoire uniforme** : c'est précisément le risque de *shortcut learning* que nous analyserons en partie 6.

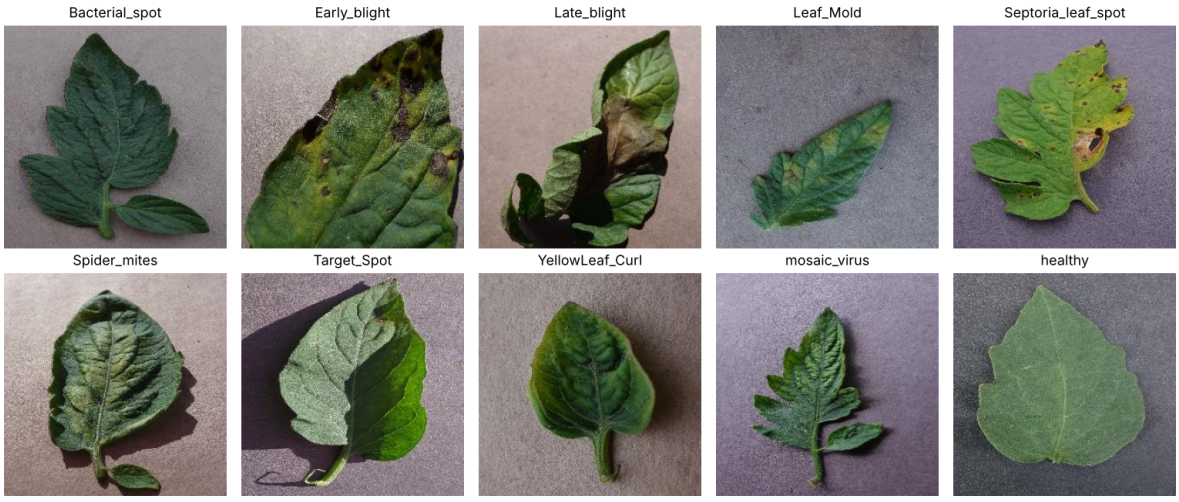


Figure 1 – Un exemple de feuille pour chacune des 10 classes (fond de laboratoire uniforme).

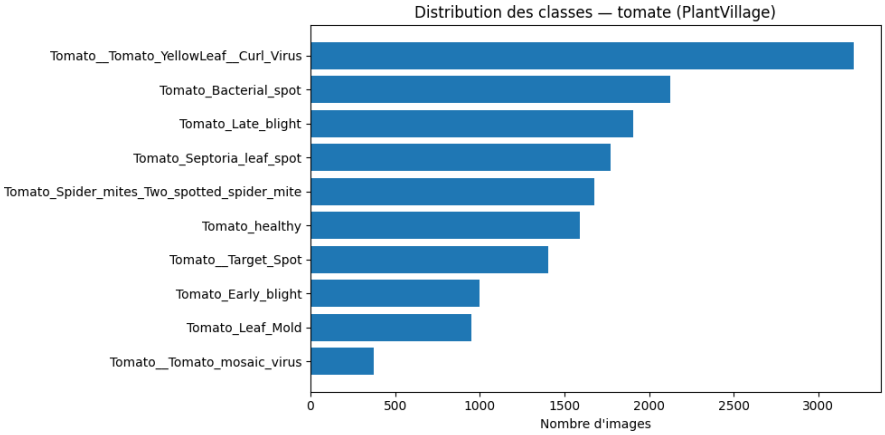


Figure 2 – Distribution des effectifs par classe (jeu complet).

Pour la normalisation, nous calculons la moyenne et l'écart-type par canal **sur le train uniquement**, afin d'éviter toute fuite d'information du test vers l'entraînement. Nous obtenons une moyenne de $[0{,}451; 0{,}465; 0{,}407]$ et un écart-type de $[0{,}178; 0{,}156; 0{,}193]$ pour les canaux RVB. Ces statistiques servent pour les CNN entraînés de zéro. Pour le transfer learning, nous utilisons au contraire les statistiques ImageNet, car les backbones pré-entraînés attendent la même normalisation que lors de leur pré-entraînement. Pour les auto-encodeurs et le VAE, nous n'appliquons aucune normalisation et gardons les pixels dans $[0, 1]$, puisque la sortie de ces modèles est une sigmoïde bornée dans $[0, 1]$.

2.3 Découpage et vérification anti-fuite

Le protocole imposé est un découpage 70% apprentissage, 15% validation, 15% test, stratifié par classe, avec une graine fixée à 42. Nous obtenons 11 197 images de train, 2 400 de validation et 2 400 de test.

Le point délicat est la fuite de données : certaines versions de PlantVillage contiennent des variantes augmentées (rotations, miroirs) d'une même feuille, qui pourraient se retrouver à la fois en entraînement et en test. Notre stratégie est la suivante. Nous calculons un *perceptual hash* (pHash) de chaque image, avec normalisation d'orientation, de sorte que des quasi-doublons partagent le même hash. Les images de même hash reçoivent le même identifiant de groupe, et le découpage est effectué **par groupe** : toutes les images d'un même groupe tombent dans le même sous-ensemble. Nous avons ainsi détecté et retiré 14 doublons exacts (16 011 vers 15 997), et nous garantissons par construction qu'aucune feuille n'apparaît sous deux formes dans deux sous-ensembles différents. Le découpage est figé dans un fichier `split.csv` rechargé à l'identique par tous les notebooks, ce qui rend les résultats directement comparables d'une partie à l'autre. Le jeu de test n'est utilisé qu'une seule fois par modèle final, jamais pour choisir un hyperparamètre ni un seuil.

2.4 Reproductibilité et budget de calcul

Toutes les graines (Python, NumPy, PyTorch, cuDNN en mode déterministe) sont fixées à 42. Les versions des bibliothèques sont listées dans `requirements.txt`, et chaque notebook s'exécute de bout en bout. L'environnement est Google Colab avec un GPU T4 gratuit, et chaque entraînement

tient en quelques dizaines de minutes. Les poids et les figures sont sauvegardés sur Google Drive (dossiers `models/` et `results/`), car les sessions gratuites s'interrompent.

3 Partie 1 : modèles de base entraînés de zéro

3.1 Méthode

Nous comparons deux CNN de même profondeur (6 couches convolutives), entraînés de zéro en 128×128.

CNN-Plain. Six blocs `Conv(3×3, padding 1)` → `BatchNorm` → `ReLU` → `MaxPool(2)`, suivis d'une tête dense $1024 \to 256 \to 10$ avec un Dropout de 0,5. Nos choix se justifient ainsi. Le noyau 3×3 est le plus petit champ réceptif utile ; empilé, il remplace de gros noyaux pour moins de paramètres. Le padding 1 conserve la taille spatiale, seul le MaxPool la divise par deux. Le nombre de filtres croît (32 vers 256) pour compenser la perte de résolution par de la richesse sémantique. La BatchNorm stabilise l'entraînement, et le Dropout sur la tête dense combat le sur-apprentissage.

Le calcul des dimensions à la main donne, pour une entrée $(3, 128, 128)$: après chaque bloc, la taille passe à $64, 32, 16, 8, 4$ puis 2, avec les canaux $32, 64, 128, 128, 256, 256$. Le vecteur aplati vaut donc $256 \times 2 \times 2 = 1024$, ce que confirme `torchinfo`. Le réseau compte 1 392 778 paramètres (environ 5,6 Mo).

CNN-Res. Même profondeur (2 convolutions par bloc, 3 blocs), mais avec des connexions résiduelles (`out + shortcut(x)`) et un Global Average Pooling à la place du `Flatten + Dense`. Le raccourci donne une voie directe au gradient, et le GAP supprime la grosse couche dense ($\approx$ 262k paramètres), ce qui réduit le sur-apprentissage à budget comparable. Le réseau compte 1 190 922 paramètres (environ 4,8 Mo), soit 14% de moins que CNN-Plain, mais environ 628 M d'opérations multiplier-additionner contre 232 M, car il garde des cartes spatiales plus grandes plus longtemps.

Les deux modèles utilisent la perte `CrossEntropyLoss`, l'optimiseur Adam ($lr = 10^{-3}$) et un early-stopping de patience 5 sur la perte de validation. Nous gardons les meilleurs poids.

3.2 Résultats

Les courbes d'apprentissage (figures 3 et 4) montrent une convergence saine, sans sur-apprentissage marqué. CNN-Plain s'arrête à l'époque 14 (meilleure perte de validation 0,1204), CNN-Res à l'époque 22 (0,0577). Au test, CNN-Plain obtient une accuracy de 0,960 et une F1-macro de 0,952 ; CNN-Res obtient 0,975 et 0,973. Les matrices de confusion (figures 5 et 6) montrent que les confusions résiduelles se concentrent sur des maladies visuellement proches (*Early_blight* contre *Late_blight*).

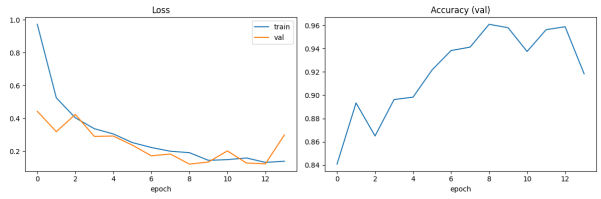


Figure 3 – CNN-Plain, courbes de perte et d'accuracy.

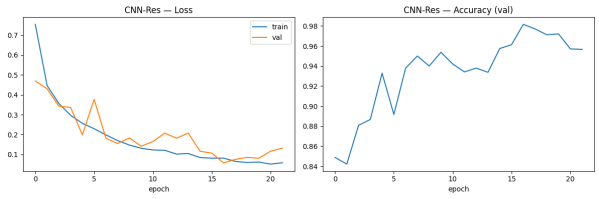


Figure 4 – CNN-Res, courbes de perte et d'accuracy.

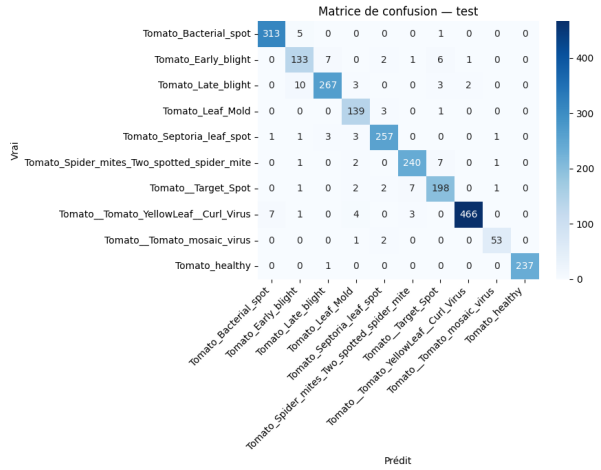


Figure 5 – CNN-Plain, matrice de confusion (test).

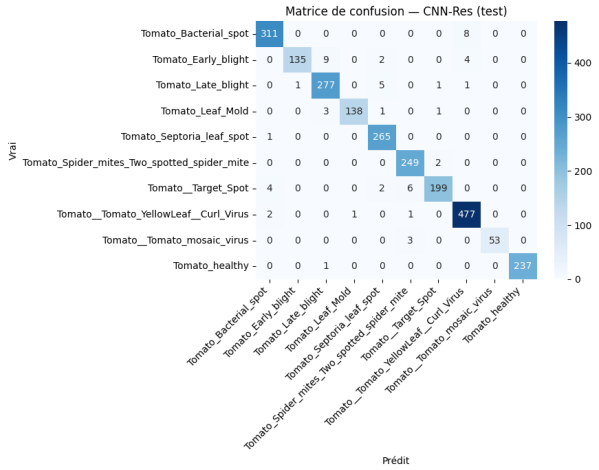


Figure 6 – CNN-Res, matrice de confusion (test).

Étude du learning rate. Nous faisons varier le learning rate sur trois valeurs, sur CNN-Plain et à horizon court (10 époques). La meilleure accuracy de validation est de 0,910 pour 10^{-2}, 0,946 pour 10^{-3} et 0,968 pour 10^{-4} (figure 7). La valeur 10^{-2} est trop grande et rend l'optimisation instable. La valeur 10^{-4} est très stable mais lente ; elle gagne sur 10 époques justement parce qu'elle ne dépasse jamais le minimum. La valeur 10^{-3} est plus bruitée à court terme mais converge plus vite vers un meilleur optimum sur un entraînement complet, d'où notre choix.

Diagnostic des gradients. Nous relevons la norme L2 du gradient de chaque couche convolutive pendant l'entraînement (figure 8). Contrairement à l'intuition, nous n'observons pas de *vanishing gradient* : aucune couche ne tombe à zéro, et la norme est même plus forte près de l'entrée. Deux raisons l'expliquent. D'une part, six couches sont trop peu pour que le phénomène apparaisse (il mord sur des réseaux de 30 à 50 couches ou plus). D'autre part, la BatchNorm, présente dans les deux réseaux, est un stabilisateur de gradient reconnu et fait déjà ce travail. À cette profondeur, les connexions résiduelles n'apportent donc ni meilleure circulation du gradient, ni gain décisif de précision.

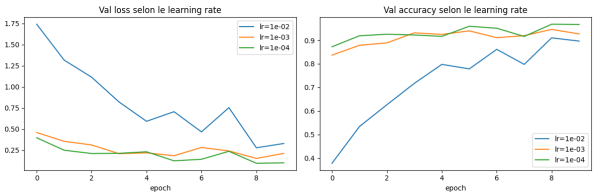


Figure 7 – Étude du learning rate (validation).

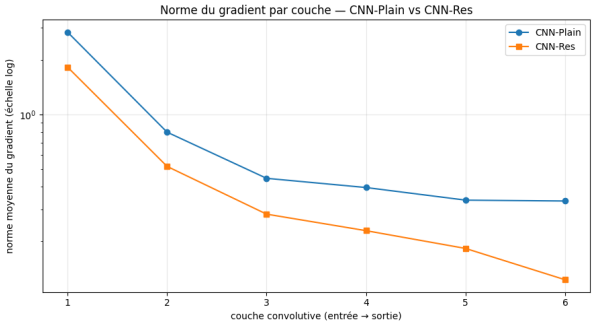


Figure 8 – Norme du gradient par couche.

3.3 Discussion et tableau comparatif

Table 2 – Comparatif des deux CNN entraînés de zéro.

Modèle	Paramètres	Taille	Temps	Accuracy	F1-macro
CNN-Plain	1,39 M	≈5,6 Mo	≈7 min (14 ép.)	0,960	0,952
CNN-Res	1,19 M	≈4,8 Mo	≈11 min (22 ép.)	0,975	0,973

Sur ce run, CNN-Res fait un peu mieux que CNN-Plain (F1-macro 0,973 contre 0,952) tout en ayant 14% de paramètres en moins, au prix d'environ trois fois plus de calcul. Le gain se concentre sur les classes visuellement difficiles. Il faut toutefois le nuancer : à seulement six couches, cet écart reste modeste et en partie dans la variabilité entre runs, et le diagnostic des gradients ne montre aucun *vanishing gradient*. L'avantage du CNN-Res vient donc surtout du GAP (moins de sur-apprentissage) et non d'une meilleure circulation du gradient. Le vrai bénéfice des connexions résiduelles n'apparaîtrait que sur des réseaux bien plus profonds.

4 Partie 2 : transfer learning et choix d'architecture

4.1 Méthode

Nous passons à des backbones pré-entraînés sur ImageNet, en 160×160 avec la normalisation ImageNet. Nous choisissons un modèle léger et un modèle lourd, aux idées opposées.

MobileNetV2 (environ 2,24 M paramètres, 8,9 Mo) repose sur les convolutions séparables en profondeur (depthwise puis pointwise), qui factorisent le filtrage spatial et la combinaison des canaux et divisent le coût de calcul par un facteur 8 à 9 à capacité comparable, ainsi que sur des blocs résiduels inversés. Il est conçu pour l'embarqué, ce qui colle à notre cible smartphone.

ResNet-50 (environ 23,53 M paramètres, 94 Mo) repose sur des connexions résiduelles profondes (50 couches) qui, par le raccourci identité, évitent le *vanishing gradient* et permettent d'empiler beaucoup de couches pour une précision maximale.

Nous comparons trois stratégies sur MobileNetV2 : (a) extraction de features (backbone gelé, nouvelle tête), (b) fine-tuning des dernières couches, (c) fine-tuning complet avec learning rate différencié (10^{-4} sur le backbone, 10^{-3} sur la tête neuve). ResNet-50 sert de second backbone sur

la stratégie (a). La latence est mesurée en batch 1 (une photo à la fois), en moyenne sur 120 passes avec warm-up, sur la machine Colab (GPU T4, CPU 2 cœurs logiques).

4.2 Résultats

Le tableau 3 donne les résultats de test. Le fine-tuning complet de MobileNetV2 est le meilleur (F1-macro 0,992), devant les CNN maison de la partie 1. L'extraction de features plafonne plus bas, car les features ImageNet gelées sont génériques et pas optimales sur des feuilles de laboratoire. ResNet-50 gelé est le plus faible et environ 4,6 fois plus lent sur CPU : un gros backbone n'apporte rien ici.

Table 3 – Transfer learning et baselines, résultats de test et latence.

Modèle	Params	Accuracy	F1-macro	Lat. CPU	Lat. GPU
MobileNetV2 (c) complet	2,24 M	0,991	**0,992**	13,9 ms	5,10 ms
MobileNetV2 (b) dern. couches	2,24 M	0,988	0,985	13,7 ms	5,13 ms
CNN-Res (128px)	1,19 M	0,975	0,973	15,4 ms	1,55 ms
CNN-Plain (128px)	1,39 M	0,960	0,952	14,1 ms	1,28 ms
MobileNetV2 (a) extraction	2,24 M	0,929	0,916	14,0 ms	5,16 ms
ResNet-50 (a) extraction	23,53 M	0,919	0,904	64,2 ms	5,39 ms

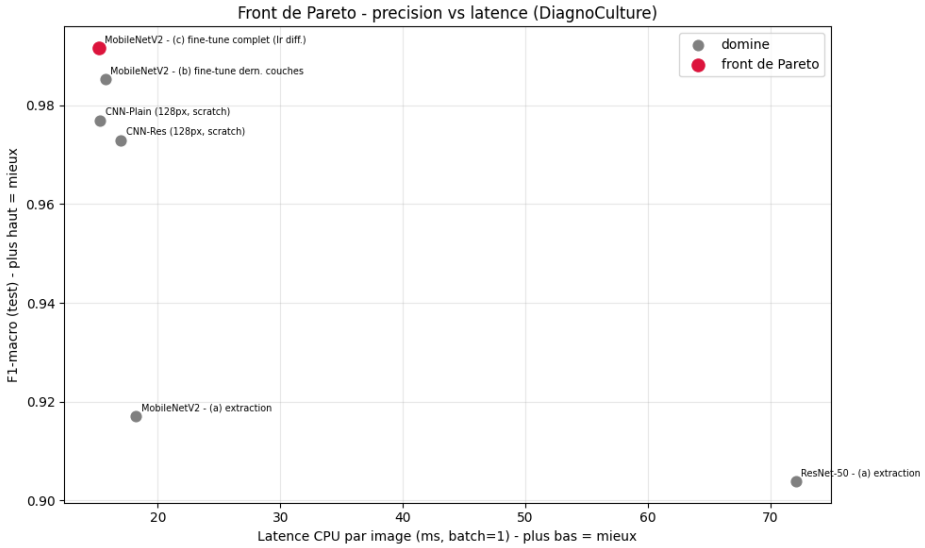


Figure 9 – Front de Pareto, F1-macro contre latence CPU.

4.3 Discussion et recommandation

Le front de Pareto (figure 9) se réduit à MobileNetV2 (b) et (c) : aucun autre modèle n'est à la fois plus précis et plus rapide qu'eux. Les modèles légers sont tous groupés en latence CPU (13,7 à 15,4 ms), le choix se jouant donc sur la précision. ResNet-50 est doublement dominé.

Pour un smartphone d'entrée de gamme, qui exécute le modèle sur CPU, les deux métriques décisives sont la latence CPU et la taille. Nous recommandons **MobileNetV2 fine-tuné complètement (stratégie c)** : c'est le meilleur F1-macro (0,992), pour une latence et une taille

quasi identiques à la stratégie (b). Si le stockage est très contraint, CNN-Plain reste le plus léger (5,6 Mo) mais au prix d'environ 4 points de F1-macro, et sans dépendre de poids ImageNet. Une nuance pour la soutenance : sur GPU, les CNN maison sont en fait plus rapides (environ 1,3 ms contre 5,1 ms), car les convolutions depthwise de MobileNet exploitent mal le parallélisme (elles sont *memory-bound*). « Léger en opérations » ne veut donc pas dire « rapide sur GPU », ce qui confirme que la métrique pertinente pour un smartphone est bien la latence CPU.

5 Partie 3 : auto-encodeurs et détection d'anomalies

5.1 Méthode

Le principe change : on n'entraîne plus à classer, mais à reconstruire des feuilles **saines uniquement**. Une feuille malade, jamais vue à l'entraînement, se reconstruit mal, et l'erreur de reconstruction $\|x - \hat{x}\|^2$ sert de score d'anomalie. L'auto-encodeur convolutif travaille en 64×64 : encodeur de 3 convolutions stride-2 (64 vers 8 pixels) et goulot linéaire de dimension k, décodeur symétrique en `ConvTranspose2d` avec sortie sigmoïde. La perte est la MSE.

5.2 Résultats

Dimension latente. La MSE de validation décroît quand k augmente : 0,00769 pour $k = 16$, 0,00544 pour $k = 64$, 0,00470 pour $k = 256$. Les reconstructions deviennent un peu plus nettes (figure 11), mais l'écart reste subtil à l'œil. Attention toutefois : la plus petite MSE n'est pas le meilleur détecteur d'anomalies, car un auto-encodeur trop performant reconstruit aussi bien les malades.

Variante débruiteuse. Nous bruitons l'entrée ($x + \mathcal{N}(0, 0{,}1)$) en gardant la cible propre. Un premier essai à *noise_std* = 0,3 détruisait l'image et donnait un modèle inutile ; corrigé à 0,1, l'entraînement se déroule normalement (meilleure MSE de validation 0,00551). Le niveau de bruit est donc un hyperparamètre critique du débruiteur. La figure 10 montre que le débruiteur retrouve bien une feuille propre à partir d'une entrée bruitée.

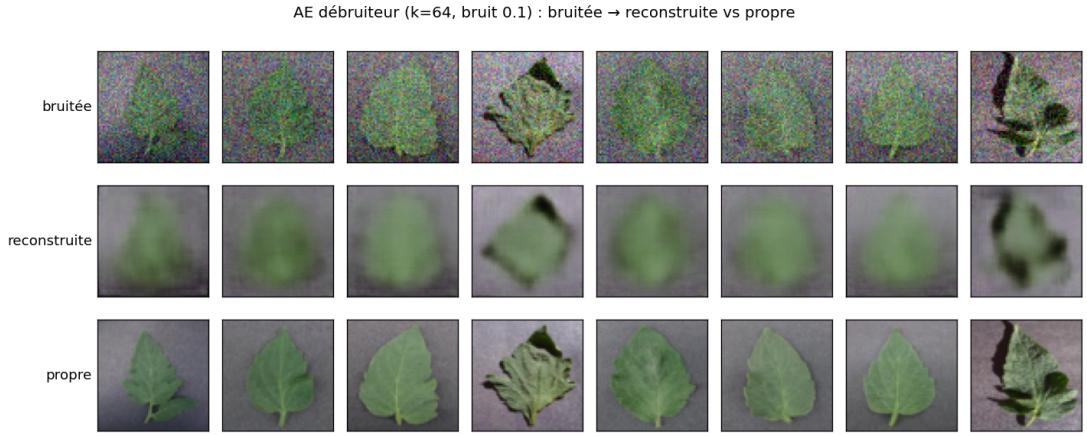


Figure 10 – Auto-encodeur débruiteur : entrée bruitée (haut) et reconstruction propre (bas).

Sélection par AUROC. Nous choisissons l'auto-encodeur retenu par l'AUROC de validation, et non par la MSE. Les AUROC sont très proches (0,646 pour le débruiteur $k = 64$, 0,646 pour $k = 16$, 0,639 pour $k = 64$ simple, 0,628 pour $k = 256$). Le fait marquant est que $k = 256$, qui a la plus petite MSE, obtient le pire AUROC : c'est la démonstration du phénomène de « reconstruction trop bonne ». Nous retenons le débruiteur $k = 64$.

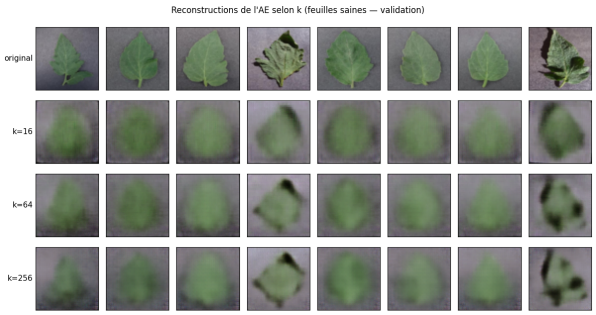


Figure 11 – Reconstructions selon k.

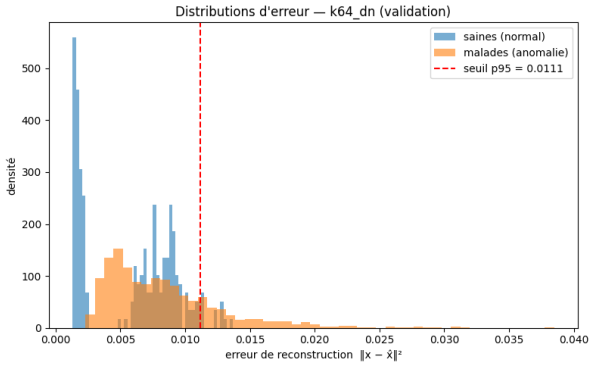


Figure 12 – Histogrammes saines contre malades.

Détection sur le test. Le seuil est fixé au 95e percentile des erreurs des feuilles saines de validation, soit 0,0111. Sur le test, nous obtenons une AUROC de 0,615, un rappel des maladies de 0,206 et un taux de fausses alarmes de 0,063. Les deux distributions se chevauchent largement (figure 12). Seules environ 21% des maladies dépassent le seuil : l'auto-encodeur reconstruit trop bien la majorité des feuilles malades.

5.3 Discussion, analyse des échecs

Le taux de détection varie fortement selon la maladie (figure 13). Sont bien détectées les maladies à symptômes étendus ou à déformation globale : Septoria (0,37), Late_blight (0,35), Early_blight (0,30), YellowLeaf_Curl (0,28). Sont mal détectées les maladies à symptômes petits, ponctuels ou subtils : Bacterial_spot (0,01), mosaic_virus (0,02), Spider_mites (0,05), Target_Spot (0,13), Leaf_Mold (0,15). La règle qui se dégage est que l'auto-encodeur capte les anomalies globales mais pas les anomalies locales fines, limite structurelle d'un score d'erreur moyenné sur toute l'image. C'est l'argument central en faveur du système hybride de la partie 4.

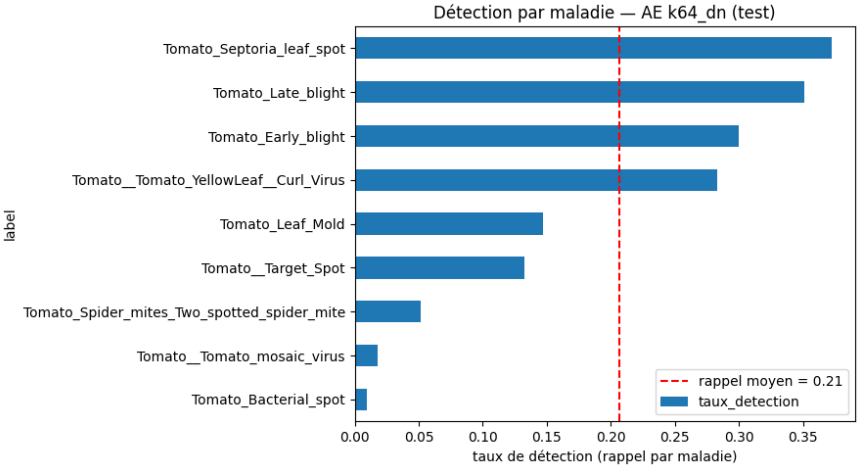


Figure 13 – Taux de détection par classe (auto-encodeur, test).

6 Partie 4 : système hybride avec rejet de l'inconnu

6.1 Méthode

C'est le cœur du projet. Nous utilisons un protocole *leave-one-class-out* : nous retirons une maladie M de l'entraînement du classifieur (elle devient l'inconnu), nous entraînons un MobileNetV2 sur les 9 classes restantes (réindexées 0 à 8), puis nous mesurons la capacité de chaque méthode à rejeter M. Nous choisissons $M = \textit{Late_blight}$ (mildiou) : maladie majeure et répandue, donc cas crédible d'émergence, et visuellement proche de l'*Early_blight*, ce qui en fait le scénario de rejet le plus exigeant. Le classifieur atteint une F1-macro de validation de 0,993.

Nous combinons deux signaux. La probabilité softmax maximale (MSP) est un signal discriminatif : sur une entrée inconnue, le classifieur est souvent moins sûr, donc MSP basse. L'erreur de reconstruction de l'auto-encodeur de la partie 3 est un signal génératif. Les seuils sont calés sur le connu de validation, avec un budget de 5% de faux rejet : τ_{MSP} au 5e percentile des MSP connues, τ_{AE} au 95e percentile des erreurs connues.

6.2 Résultats

Hybride naïf. Une première règle « rejet si au moins un signal s'alarme » (max des deux scores normalisés) échoue : l'AUROC tombe à 0,827, sous la MSP seule (0,944). L'auto-encodeur flague toutes les maladies, connues comme inconnues, et ajoute surtout du bruit de faux rejet.

Correction par veto. Nous ne laissons l'auto-encodeur voter que sur une erreur extrême, au-delà du 99e percentile des feuilles connues ($\tau_{ae}^{hi} = 0{,}0201$). La MSP reste le signal principal. Le tableau 4 montre que l'hybride égale alors la MSP sans jamais la dégrader.

Table 4 – Rejet de l'inconnu M (test), veto au 99e percentile.

Méthode	AUROC	Rejet de M	Faux rejet connu	Acc. connu accepté
MSP	0,944	0,681	0,046	0,999
AE seul	0,562	0,119	0,047	0,995
Hybride	0,944	0,670	0,046	0,999

Couverture et précision. En balayant tous les seuils (figure 14), MSP et hybride sont confondues et hautes (précision proche de 1 tant que la couverture reste sous 0,6). Au point de fonctionnement à 5% de faux rejet, on obtient une couverture d'environ 0,87 et une précision d'environ 0,955 : on automatise environ 87% des cas à plus de 95% de fiabilité, et on route les 13% restants vers un expert.

Jeux hors-distribution. Sur l'OOD proche (300 feuilles d'autres espèces issues de PlantVillage), les taux de rejet sont de 77,7% pour la MSP, 19,3% pour l'AE, 80,0% pour l'hybride. Sur l'OOD lointain (41 photos smartphone de non-feuilles : main, gobelet, mur, sandales), ils sont de 92,7%, 43,9% et 92,7% (figure 15). Contrairement à notre hypothèse de départ, l'auto-encodeur n'écrase pas la MSP sur le non-feuille : une surface lisse (mur, main) se reconstruit avec une erreur faible, car l'auto-encodeur note la reconstruction pixel, aveugle à la sémantique.

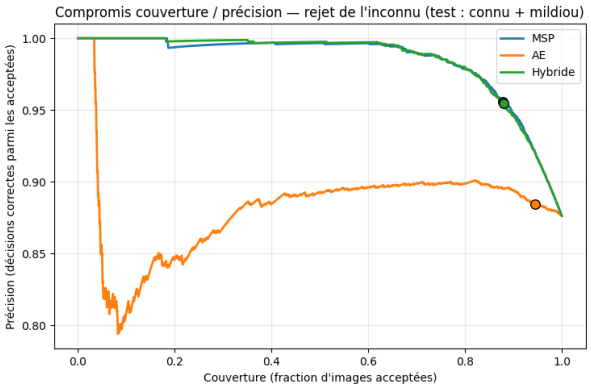


Figure 14 – Courbe couverture / précision.

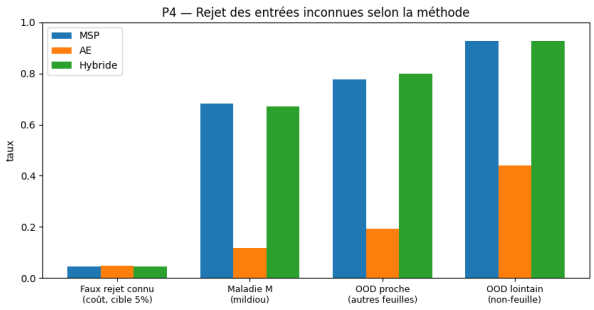


Figure 15 – Taux de rejet par méthode et par régime.

6.3 Discussion

La confiance du classifieur (MSP) est le signal de rejet le plus universel : elle domine sur la maladie inconnue et sur les deux régimes OOD. L'auto-encodeur est un détecteur faible et, ici, redondant, car son erreur moyennée est aveugle à la sémantique. La valeur de l'hybride n'est pas un gain de points mais la robustesse : avec le veto conservateur, il n'est jamais pire que la MSP et sert de filet de sécurité gratuit contre le non-feuille extrême. Notre hypothèse initiale (l'auto-encodeur gagnant sur le lointain) est réfutée par les données, ce qui constitue un meilleur résultat scientifique, car il explique pourquoi un détecteur génératif par reconstruction ne complète pas un classifieur discriminatif sur ce problème.

7 Partie 5 : extension générative (VAE)

7.1 Méthode

Nous choisissons l'option B (auto-encodeur variationnel) plutôt que le DCGAN, pour réutiliser l'infrastructure de la partie 3 et pour prolonger la comparaison des scores d'anomalie, et parce qu'un GAN serait plus instable sur un délai court. Un VAE envoie chaque image sur une petite zone de l'espace latent (une gaussienne de moyenne μ et variance σ^2) plutôt que sur un point. En forçant ces zones à se tasser autour de l'origine (terme KL), l'espace latent devient continu et sans trous, ce qui permet de générer. Le `ConvVAE` reprend l'encodeur et le décodeur de la partie 3, avec deux têtes (μ et $\log \sigma^2$) et la reparamétrisation $z = \mu + \sigma\varepsilon$. Il compte 874 595 paramètres. La perte est l'ELBO (reconstruction MSE plus terme KL), avec $\beta = 1$.

7.2 Résultats

L'entraînement (60 époques, environ 2 min sur T4) atteint une meilleure perte de validation de 104,9 (figure 16). Le KL ne s'effondre pas et se stabilise autour de 22 : il n'y a pas de *posterior collapse*, le modèle utilise réellement son espace latent. Les images générées par échantillonnage du prior $\mathcal{N}(0, I)$ sont des feuilles saines reconnaissables mais un peu floues (figure 18), flou attendu d'un VAE. L'interpolation linéaire entre deux feuilles reste réaliste sur tout le chemin (figure 17), preuve de la continuité de l'espace latent : un auto-encodeur classique échouerait ici, le milieu du chemin tombant souvent sur du bruit. La projection t-SNE colorée par classe (figure 19) montre une organisation structurée.

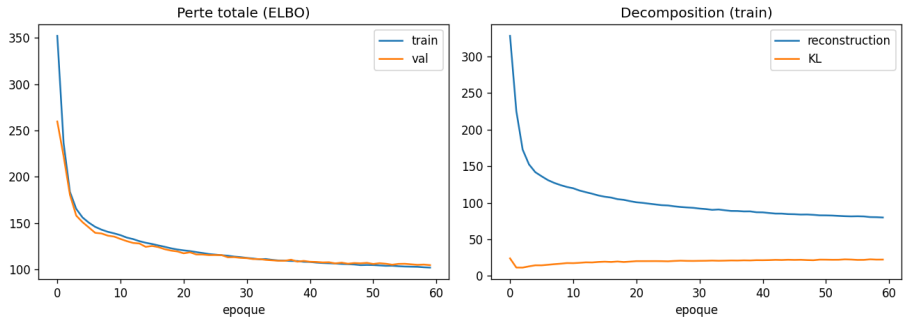


Figure 16 – VAE, courbes de perte (reconstruction et KL) en entraînement et validation.

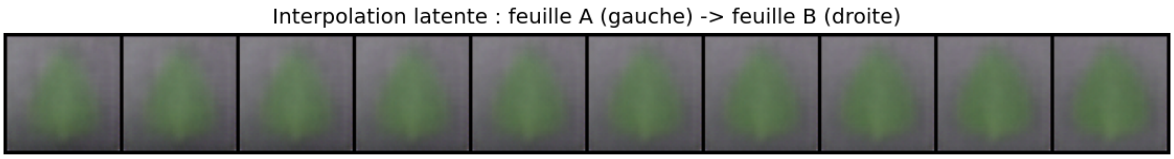


Figure 17 – Interpolation linéaire dans l'espace latent entre deux feuilles : chaque image intermédiaire reste une feuille plausible.

Comme score d'anomalie, le VAE fait légèrement mieux que l'auto-encodeur simple : AUROC de 0,663 (reconstruction) et 0,678 (ELBO), contre 0,615 pour l'auto-encodeur de la partie 3 recalculé sur le même pipeline (tableau 5). L'ELBO est le meilleur score, car une image malade tombe

dans une zone moins typique de l'espace latent, ce que le terme KL capte en plus de l'erreur de reconstruction. Le gain reste toutefois modeste et les distributions se chevauchent toujours.

Table 5 – Score d'anomalie, auto-encodeur contre VAE (même test).

Méthode	AUROC
Auto-encodeur simple (P3)	0,615
VAE, reconstruction	0,663
VAE, ELBO (recon + KL)	**0,678**

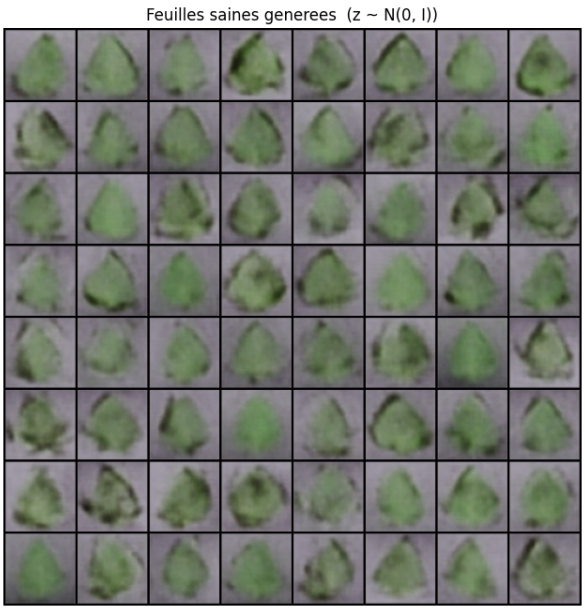


Figure 18 – Images générées par le VAE.

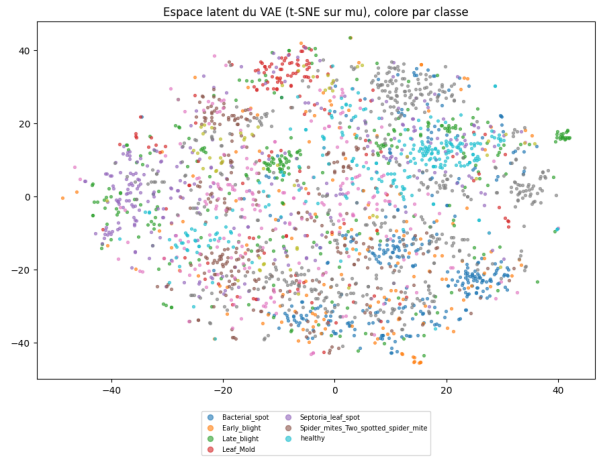


Figure 19 – Espace latent (t-SNE) coloré par classe.

7.3 Discussion

Le VAE apporte surtout une capacité générative et une amélioration marginale de la détection, sans résoudre le problème de fond : comme en partie 3, un score fondé sur la reconstruction moyennée rate les lésions petites et locales. Les images générées reproduisent aussi le fond de laboratoire uniforme, ce que le VAE a encodé comme faisant partie de la feuille, illustration directe du biais de fond à discuter en partie 6.

8 Partie 6 : analyse critique et responsabilité

8.1 Explicabilité par Grad-CAM

Nous produisons des cartes Grad-CAM pour le meilleur classifieur (MobileNetV2 complet, F1-macro 0,992), sur des prédictions correctes et incorrectes, en accrochant la dernière couche convolutive (figures 20 et 21). Le résultat principal est rassurant : le modèle ne s'appuie pas sur le fond. Sur toutes les images, le fond uniforme est systématiquement ignoré (en bleu) et la chaleur reste sur la feuille. Le risque de *shortcut learning* redouté avec PlantVillage n'est donc pas réalisé ici. Quand le modèle réussit, la chaleur se concentre sur la zone pertinente (lésion nécrotique, surface

mouchetée). Quand il échoue, l'attention reste sur la feuille mais devient diffuse et centrale, et rate les petites taches diagnostiques des bords. Le risque principal de ce prototype n'est donc pas le fond, mais une dépendance à l'apparence globale au détriment des symptômes locaux discrets, la même limite que pour l'auto-encodeur et le VAE.

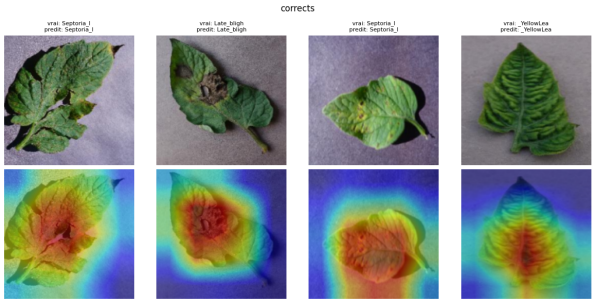


Figure 20 – Grad-CAM, prédictions correctes.

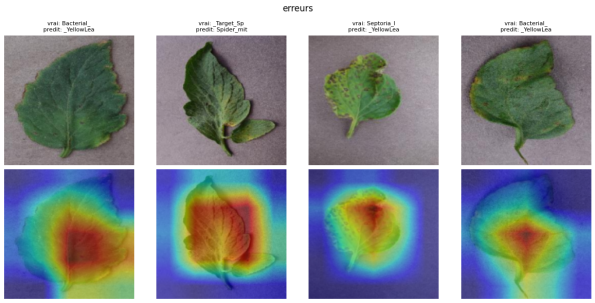


Figure 21 – Grad-CAM, prédictions incorrectes.

8.2 Analyse d'erreurs

Le modèle ne commet que 21 erreurs sur 2 400 images de test (accuracy d'environ 99,1%). Une confusion systématique domine : *Bacterial_spot* vers *YellowLeaf_Curl* (8 cas, soit environ 38% des erreurs). Les petites taches brunes de Bacterial_spot, souvent sur les bords, sont manquées (comme le confirme le Grad-CAM, attention diffuse au centre) ; quand la feuille jaunit, le modèle se rabat sur YellowLeaf_Curl, la classe majoritaire. Au total, 10 des 21 erreurs (environ 48%) prédisent YellowLeaf_Curl, nette manifestation du déséquilibre : en cas de doute, la classe dominante l'emporte. Les autres erreurs sont des confusions entre maladies visuellement proches, à raison d'un cas chacune (*Early_blight* contre *Late_blight* dans les deux sens, *Spider_mites* contre *Target_Spot*, *Leaf_Mold* vers *Spider_mites*, *Septoria* vers *Bacterial_spot*). On compte enfin un cas *mosaic_virus* vers *Early_blight* (la plus petite classe, difficile à apprendre) et un faux positif *healthy* vers *Early_blight* (feuille saine déclarée malade), important pour la discussion éthique. Les erreurs se concentrent donc sur des symptômes petits, locaux ou peu contrastés, exactement les maladies les moins bien détectées par l'auto-encodeur.

8.3 Du laboratoire au champ

PlantVillage est pris en laboratoire sur fond uniforme, éclairage contrôlé et feuille isolée. Une photo prise dans un champ des Niayes diffère sur plusieurs points : fond non uniforme (terre, autres feuilles, mains), éclairage variable (soleil, ombres), résolution et cadrage irréguliers, et variétés locales absentes du jeu. Le Grad-CAM est plutôt rassurant sur le fond, mais le changement de distribution resterait un risque réel. Pour y remédier, nous collecterions un jeu « terrain » : plusieurs centaines de photos de feuilles prises au smartphone directement dans les parcelles, sur fond naturel, à différentes heures et saisons, étiquetées par un agent agricole, en incluant volontairement des cas difficiles (début de maladie, co-infections) et des images non-feuille pour renforcer le rejet.

8.4 Enjeux éthiques et sociaux

Les deux types d'erreur n'ont pas le même coût. Un faux négatif (maladie déclarée saine) retarde le traitement et laisse la maladie se propager, perte potentiellement lourde pour le maraîcher. Un

faux positif (feuille saine déclarée malade) déclenche un traitement inutile, avec un coût financier et environnemental (pesticides). Le système doit donc rester un outil d'aide, et non de décision automatique : la place de l'humain (agent agricole, encadrant de la coopérative) est centrale, surtout pour les cas rejetés comme « inconnu ». Enfin, l'accessibilité impose un fonctionnement hors-ligne sur un smartphone modeste, ce qui justifie le choix d'un modèle léger (MobileNetV2) et la possibilité d'un export quantifié.

8.5 Limites et perspectives

Les limites principales sont la dépendance aux symptômes globaux (les lésions fines sont mal vues), l'écart laboratoire / champ non encore testé sur un vrai jeu terrain, et un détecteur d'anomalies faible (AUROC d'environ 0,6) qui ne tient que grâce à la MSP. Les perspectives sont la collecte d'un jeu terrain, l'essai d'un backbone plus récent (ConvNeXt léger, Vision Transformer), un export ONNX avec quantification pour mesurer le gain de taille et de latence, et une petite application de démonstration.

9 Tableau récapitulatif global

Table 6 – Récapitulatif de toutes les expériences.

Expérience	Rôle	Métrique principale	Valeur
CNN-Plain (P1)	Classif. 10 classes	F1-macro (test)	0,952
CNN-Res (P1)	Classif. 10 classes	F1-macro (test)	0,973
MobileNetV2 (c) (P2)	Classif. 10 classes	F1-macro (test)	**0,992**
MobileNetV2 (b) (P2)	Classif. 10 classes	F1-macro (test)	0,985
MobileNetV2 (a) (P2)	Classif. 10 classes	F1-macro (test)	0,916
ResNet-50 (a) (P2)	Classif. 10 classes	F1-macro (test)	0,904
Auto-encodeur k=64 débruiteur (P3)	Détection d'anomalies	AUROC (test)	0,615
Hybride MSP + veto AE (P4)	Rejet de l'inconnu	AUROC connu/inconnu	0,944
VAE k=64 (P5)	Détection d'anomalies (ELBO)	AUROC (test)	0,678
MobileNetV2 (P6)	Explicabilité, erreurs	Accuracy (test)	0,991

10 Conclusion et questions de synthèse

Nous avons construit un prototype répondant aux trois exigences : un classifieur précis (F1-macro 0,992), léger et rapide sur CPU (MobileNetV2), et un système hybride capable de rejeter l'inconnu en automatisant environ 87% des cas à plus de 95% de fiabilité. Nous répondons maintenant aux six questions de synthèse.

1. Pourquoi les connexions résiduelles facilitent-elles l'entraînement d'un réseau profond ? Le raccourci identité offre une voie directe au gradient, qui peut traverser le réseau sans être multiplié couche après couche. Cela empêche le gradient de s'évanouir dans les couches proches

de l'entrée et laisse chaque bloc n'apprendre qu'une correction résiduelle, plus facile à optimiser. Notre partie 1 montre cependant que ce bénéfice ne se manifeste qu'en grande profondeur : à six couches, la BatchNorm suffit déjà à stabiliser les gradients, et nous n'observons pas de *vanishing gradient*.

2. Que gagne-t-on, et que perd-on, en passant du CNN de zéro au transfer learning ? On gagne en précision (F1-macro de 0,952 à 0,992) et en temps, car le backbone a déjà appris des features visuelles génériques sur ImageNet. On perd un peu en légèreté et en indépendance : MobileNetV2 est plus gros que CNN-Plain, dépend de poids externes, et sur GPU il est même plus lent à cause des convolutions depthwise. Le transfer learning n'est donc pas gratuit, mais le compromis est très favorable ici.

3. Pourquoi un classifieur softmax est-il un mauvais détecteur d'inconnu, et en quoi l'auto-encodeur apporte-t-il une information complémentaire ? Le softmax est contraint de répartir une probabilité totale de 1 sur les classes connues : il ne peut pas dire « aucune ». Il reste donc souvent confiant à tort sur une entrée inconnue. L'auto-encodeur, lui, n'a appris qu'à reconstruire des feuilles saines et apporte un signal différent (l'erreur de reconstruction), indépendant des classes. En pratique, la partie 4 montre que ce signal est faible et surtout redondant avec la MSP, mais qu'il sert de filet de sécurité contre le non-feuille extrême.

4. Quelles sont les limites d'un auto-encodeur comme détecteur d'anomalies ? D'abord la « reconstruction trop bonne » : un modèle de grande capacité reconstruit aussi bien les anomalies, ce que montre $k = 256$ (plus petite MSE mais pire AUROC). Ensuite l'aveuglement sémantique : l'erreur moyennée sur l'image rate les lésions petites et locales, et une surface lisse hors-sujet (mur, main) obtient une erreur faible. Enfin la sensibilité au fond : l'auto-encodeur encode le fond uniforme comme faisant partie de la feuille.

5. Quelle métrique est la plus pertinente, et pourquoi pas l'accuracy seule ? Le jeu est déséquilibré (rapport d'environ 8,6). L'accuracy peut rester élevée tout en ignorant les classes rares. Nous privilégions la F1-macro, qui moyenne les classes à poids égal et donc protège les classes minoritaires, complétée par le rappel par classe et la matrice de confusion. Pour la détection d'anomalies, l'AUROC est préférée car elle est indépendante du seuil.

6. Si vous deviez déployer ce système demain, quelles trois précautions prendriez-vous ? D'abord, garder l'humain dans la boucle : le système propose, l'agent agricole décide, surtout sur les cas « inconnu ». Ensuite, afficher la confiance et le rejet plutôt qu'un diagnostic sec, pour ne pas se tromper avec aplomb. Enfin, valider sur un vrai jeu terrain des Niayes avant tout usage, car l'écart laboratoire / champ n'est pas encore mesuré.

A Déclaration d'usage de l'IA générative

Conformément aux règles du module, nous déclarons avoir utilisé un assistant d'IA générative comme aide au projet. Les usages ont été les suivants : guidage et relecture du code des notebooks, aide à la correction des chemins de fichiers lors de la réorganisation du dépôt, vérification de la

cohérence entre les chiffres produits par les notebooks et le texte des commentaires, et aide à la rédaction et à la mise en forme de ce rapport. Nous avons systématiquement vérifié les éléments produits : les chiffres cités proviennent de l'exécution réelle de nos notebooks sur Google Colab (GPU T4), les figures sont celles sauvegardées dans `results/`, et nous sommes en mesure d'expliquer chaque choix de code et chaque résultat lors de la soutenance. L'assistant n'a pas été utilisé pour copier le travail d'une autre équipe.